



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

SDS II: Combinational Logic

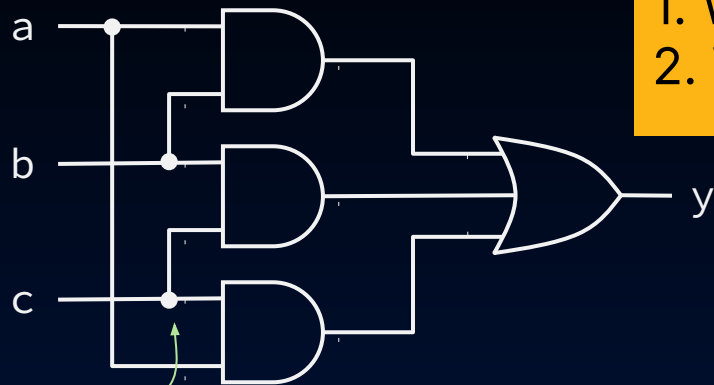


Agenda

Gate Diagram → Truth Table

- Practice
- Boolean Algebra
- Verify/Simplify Circuits
- Design Circuits:
Sum-of-Products Form

What Does This Circuit Do?



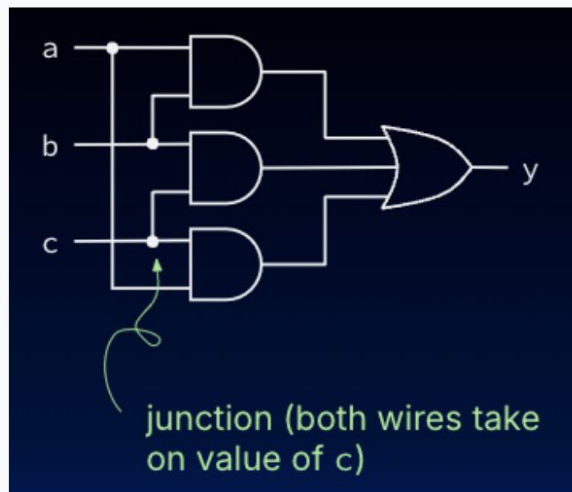
junction (both wires take
on value of c)

1. Write out the truth table for this circuit.
2. What does this circuit do, functionally?

a	b	c	y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	



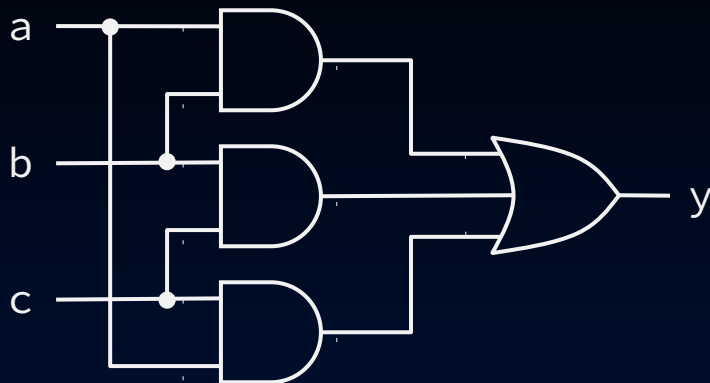
L20: What does this circuit do? (select all that apply)



- A. 1 if $a = b = 1$ 0%
- B. 1 if $b = c = 1$ 0%
- C. 1 if $a = c = 1$ 0%
- D. 1 if $a = b = c = 1$ 0%
- E. 1, if at least two of a, b, c are 1 0%
- F. The “majority” bit among a, b , and, c 0%
- None of the above 0%



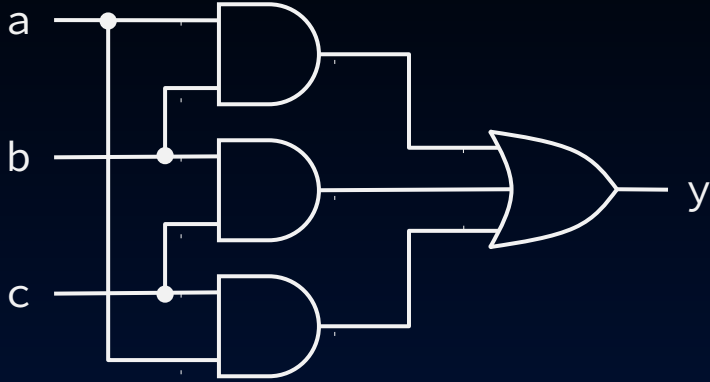
What Does This Circuit Do? Majority Circuit



What does this circuit do?
(select all that apply)

- A.** 1 if $a = b = 1$
- B.** 1 if $b = c = 1$
- C.** 1 if $a = c = 1$
- D.** 1 if $a = b = c = 1$
- E.** 1 if at least two of a, b, c are 1
- F.** The "majority" bit among $a, b,$ and c
- G.** None of the above

Majority Circuit: Boolean Expression



a	b	c	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Agenda

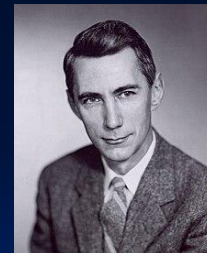
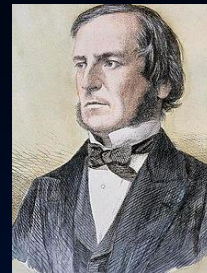
Boolean Algebra

- Practice
- Boolean Algebra
- Verify/Simplify Circuits
- Design Circuits:
Sum-of-Products Form

Historical Note: Boolean Algebra

- Boole, a 19th century mathematician, developed a mathematical system (algebra) for logic.
 - Primitive functions: AND, OR, NOT
 - Later known as Boolean Algebra
- Early 20th century computer designers noticed common patterns in their work: ANDs, ORs, ...
- Shannon, 1940, wrote his M.S. thesis that linked Boolean Algebra to logic gates.
 - There is a one-to-one correspondence between circuits composed of AND, OR, NOT gates and Boolean Algebra equations!
 - Can now apply math to give theory to hardware design, minimization, ...

George Boole
(1815–1864)



Claude Shannon
(1916–2001)

Three Basic Boolean Operations: AND, OR, NOT

AND



a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

$$a \cdot b$$

$$ab$$

OR



a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

$$a + b$$

NOT



a	y
0	1
1	0

$$\bar{a}$$

NAND, NOR, XOR
can be composed
with these
operations. Try
it/see notes!



Why use Boolean Algebra? (1/2)

- Boolean algebra means we can manipulate equations (easier) to manipulate circuits (difficult).
- Algebraic manipulation defined via **Laws of Boolean Algebra**.

Laws of Boolean Algebra

Know how to use these formulas, but no need to memorize. See reference card!

AND Form

OR Form

$$x \cdot y = y \cdot x$$

$$x + y = y + x$$

Commutativity

$$(xy)z = x(yz)$$

$$(x + y) + z = x + (y + z)$$

Associativity

$$x \cdot 1 = x$$

$$x + 0 = x$$

Identity

$$x \cdot 0 = 0$$

$$x + 1 = 1$$

Laws of 0's and 1's

$$xy + x = x$$

$$(x + y)x = x$$

Uniting Theorem

$$x(y + z) = xy + xz$$

$$x + yz = (x + y)(x + z)$$

Distributivity

$$x \cdot x = x$$

$$x + x = x$$

Idempotence

$$x \cdot \bar{x} = 0$$

$$x + \bar{x} = 1$$

Inverse (Complement)

$$\overline{(xy)} = \bar{x} + \bar{y}$$

$$\overline{(x + y)} = \bar{x} \cdot \bar{y}$$

DeMorgan's Laws

L19: Place a pin on the Boolean Algebra Law you'd like to go over. Vote once.

AND Form	OR Form	
$x \cdot y = y \cdot x$	$x + y = y + x$	Commutativity
$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	Associativity
$x \cdot 1 = x$	$x + 1 = 1$	Identity
$x \cdot 0 = 0$	$x + 0 = x$	Laws of 0's and 1's
$xy + x = x$	$(x + y)x = x$	Uniting Theorem
$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
$x \cdot x = x$	$x + x = x$	Idempotence
$x \cdot \bar{x} = 0$	$x + \bar{x} = 1$	Inverse (Complement)
$\overline{(xy)} = \bar{x} + \bar{y}$	$\overline{(x + y)} = \bar{x} \cdot \bar{y}$	DeMorgan's Laws





Scratch Space



Agenda

Verify/Simplify Circuits

- Practice
- Boolean Algebra
- Verify/Simplify Circuits
- Design Circuits:
Sum-of-Products Form

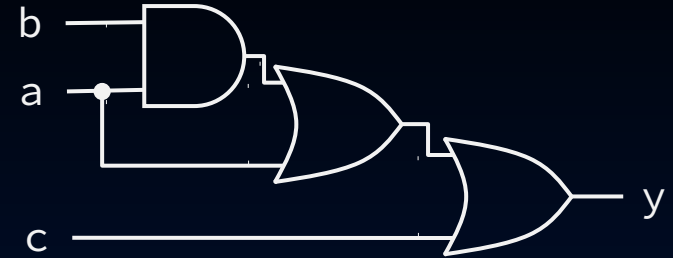


Why use Boolean Algebra? (2/2)

- Boolean algebra means we can manipulate equations (easier) to manipulate circuits (difficult).
- Algebraic manipulation defined via **Laws of Boolean Algebra**.
- Main use: **Design functionally equivalent circuits**.
 - **Verify circuits**: Are two circuits equivalent? Verify via algebraic manipulation until the expressions match
 - **Simplify circuits**: Simplify boolean expressions. Fewer gates means fewer transistors and more energy efficiency!



How to show equivalence?



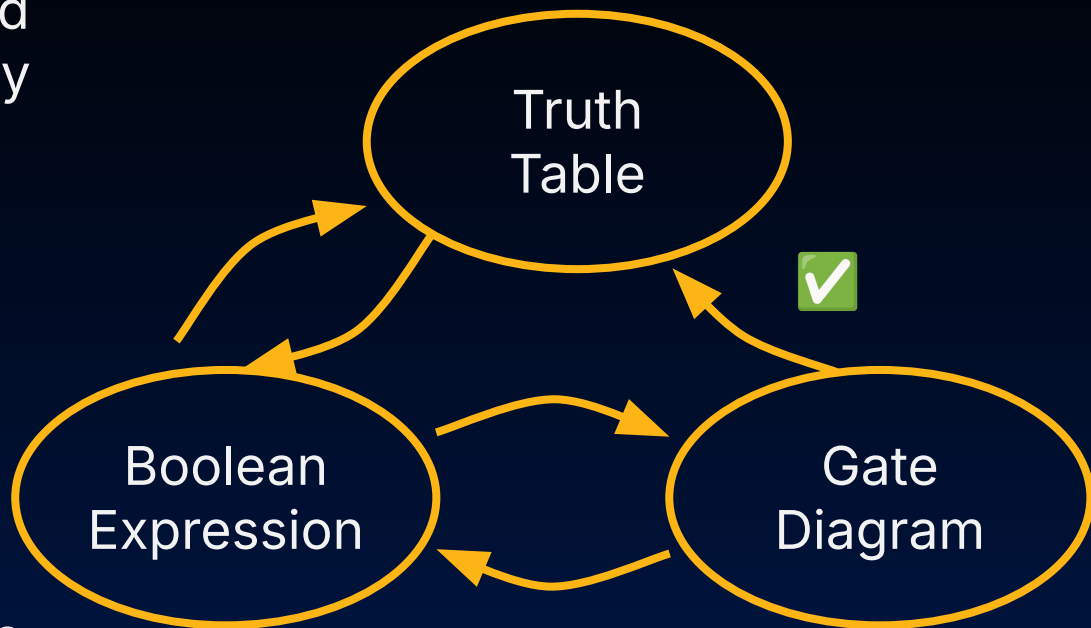


Design Circuits: Sum-of-Products Form

- Practice
- Boolean Algebra
- Verify/Simplify Circuits
- Design Circuits:
Sum-of-Products Form

Three Representations of CL Block

- Truth Table is unique and helps define functionality of a block
- Many possible:
 - Gate Diagrams
 - Boolean Expressions
- Use Boolean Algebra Laws to translate between representations.





Truth Table → Gate Diagram?

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Sum of Products, a Boolean canonical form:

- $y = 1$ if at least one of these expressions is 1
- $y = 0$ otherwise (none of these expressions are 1)

Yan, SP26

Given this truth table, how do we construct a gate diagram?

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$y = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + a\bar{b}\bar{c} + ab\bar{c}$$

Sum of Products

GOT TO HERE

Given this truth table, how do we construct a gate diagram?

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$y = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + a\bar{b}\bar{c} + ab\bar{c}$$

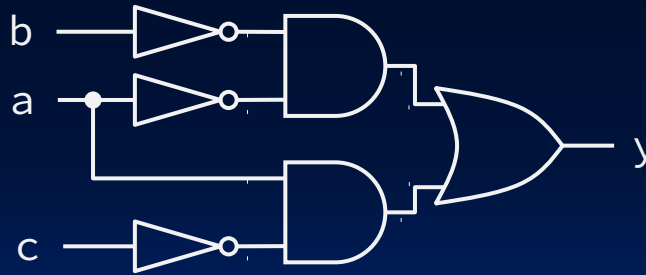
Sum of Products

...

...

$$= \bar{a}\bar{b} + a\bar{c}$$

Simplification





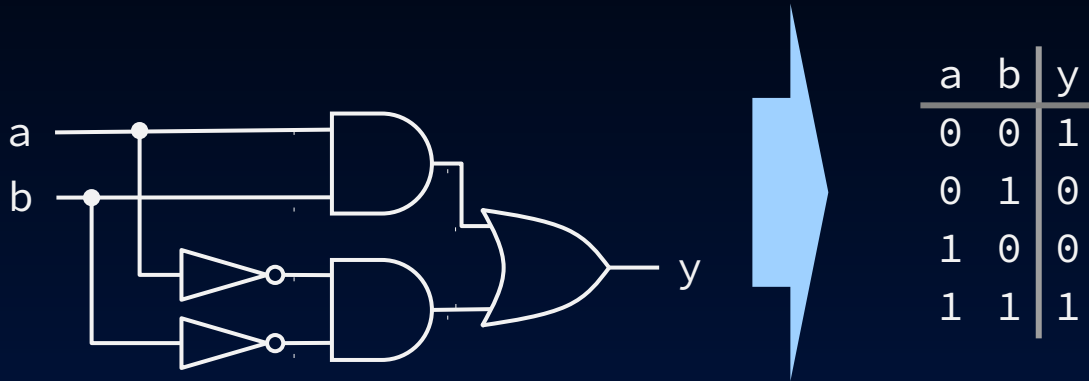
Agenda

[if time] Practice

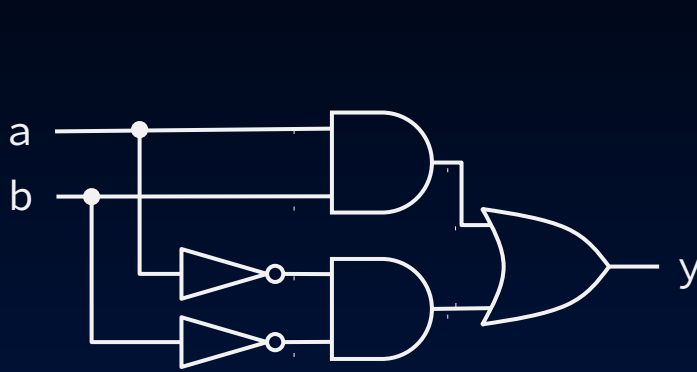
- Practice
- Boolean Algebra
- Verify/Simplify Circuits
- Design Circuits:
Sum-of-Products Form

Circuit → Truth Table (2/3)

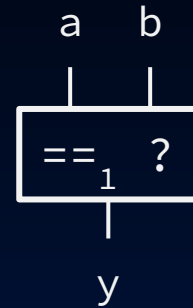
- Compare circuit from last time



- Compare circuit!



a	b	y
0	0	1
0	1	0
1	0	0
1	1	1



$y = 1$ iff a, b equal

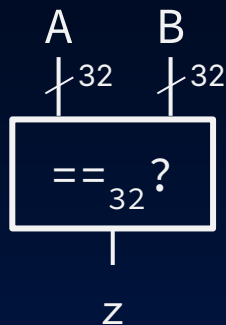
After we have designed this circuit with gates...

We can represent it as a **combinational logic block** to compose more complex circuits.



32-bit Equality Compare Circuit (1/3)

- Example: RV32I instruction `beq rs1, rs2, label`
 - If value in **rs1 == value** in rs2, then go to instruction at label, else go to next instruction.
- Somewhere in the processor must be a **circuit that compares 32-bit values**.

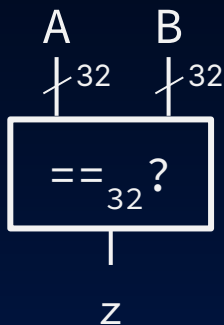


$z = 1$ iff $A == B$,
32-bit A, B; 1-bit z



32-bit Equality Compare Circuit (2/3)

- The 32-bit compare block has a truth table that is prohibitively expensive to write out by hand.



$z = 1$ iff $A == B$,
32-bit A, B; 1-bit z

a_{31}	...	a_1	a_0	b_{31}	...	b_1	b_0	z
0	...	0	0	0	...	0	0	1
0	...	0	0	0	...	0	1	0
	...				...			...
1	...	1	1	1	...	1	1	1

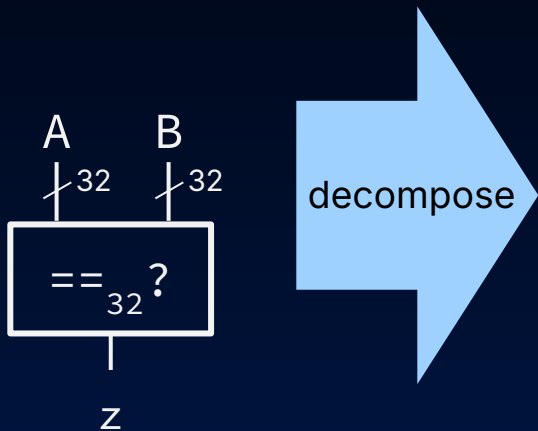
How many rows?

$$2^{32+32} = 2^{64} \text{ rows}$$

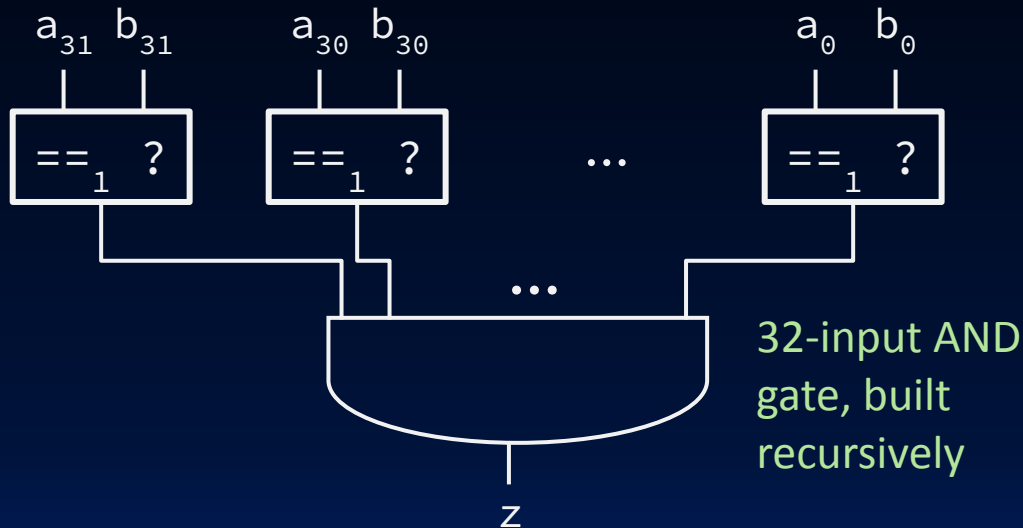
32-bit Equality Compare Circuit (3/3)

- However, we know the target functionality and can **decompose the logic** into smaller blocks.

Modular design: If truth tables are too big to construct, define smaller blocks first.



$z = 1$ iff $A == B$,
32-bit A, B ; 1-bit z



$z = 1$ iff $a_{31} == b_{31}$ AND $a_{30} == b_{30}$ AND ... AND $a_0 == b_0$

Yan, SP26